

Jupyter Notebook SQL 上机实验报告

姓名：凌晨 学号：3125351112

一、实验相关概念

Python 语言

Python 是一种通用编程语言，易学易用，广泛应用于数据分析、人工智能和 Web 开发等领域。本实验依赖 Python 语言版本 3.8 及以上。

Jupyter Notebook

Jupyter Notebook 是一个支持交互式代码编写与运行的环境，便于实时观察实验结果，同时可以将文档与代码整合在一起。

二、实验环境安装与配置

安装 Python

从 Python 官网 下载并安装 Python（3.8 及以上版本）。如果系统中已经安装了合适版本，可以跳过这一步。

配置 Jupyter Notebook

使用 Anaconda,通过安装其自带的 Jupyter Notebook，安装教程参考 Anaconda 安装教程。

配置虚拟环境 base

确保使用 Anaconda 自带的基础环境 base 来运行 Jupyter Notebook。打开终端或 Anaconda Prompt，激活 base 环境：conda activate base。确保所有依赖库安装在 base 环境中：pip install jupyter mysql-connector-python。检查环境是否正常：conda list

实验代码准备

将实验所需的代码压缩包解压到系统登录用户的目录下（C:\Users\guoyiyang）。启动 Jupyter Notebook，进入代码目录并运行实验代码文件以验证环境是否配置正确。

三、实验操作程及结果

1. 实验一

部分截图：

...	几秒前	
data	6 小时前	
ex00-Introduction.ipynb	19 小时前	6.2 kB
ex01-Quick Start.ipynb	16 小时前	5.91 kB
ex02-Query Table Information.ipynb	6 小时前	36.2 MB
ex03-Retrieving Data with SELECT.ipynb	6 小时前	90.1 kB
ex04-Constrain the Number of Rows Returned by a SELECT Query.ipynb	6 小时前	17.6 kB
ex05-Filtering a Query with WHERE.ipynb	6 小时前	141 kB
ex06-Doing Math Across Table Columns.ipynb	6 小时前	33 kB
ex07-Aggregating data with GROUP BY and ORDER BY.ipynb	6 小时前	52.4 kB
ex08-Joining Tables.ipynb	6 小时前	35.5 kB
ex09-Advanced Query Techniques of CASE and Subquery.ipynb	6 小时前	38.6 kB
ex10-Using Views to Simplify Queries.ipynb	6 小时前	15.4 kB
ex11-Dealing with NULL Values.ipynb	6 小时前	32.5 kB
ex12-Bonus from Interactive with Pandas.ipynb	5 小时前	21.1 kB
ex13-Using SQLite as a Time Series Database.ipynb	5 小时前	274 kB
s01-Transaction (Begin, Commit, Rollback).ipynb	4 年前	10 kB
s02-Accessing a SQLite Database with Pandas.ipynb	4 年前	41.5 kB
LICENSE	4 年前	1.07 kB
README.md	4 年前	1.7 kB

ex03-Retrieving Data with SELECT

When working with databases and SQL, the most common task is to request data from one or more tables, which returns this data in the form of a result table. These result tables are called result-sets. The **SELECT** statement accomplishes this. In most applications, **SELECT** is the most commonly used data query language (DQL) command. Moreover, the **SELECT** can do far more than simply retrieve and display data and I will show you in the following sections.

SELECT Syntax

- `SELECT column1, column2, columnN FROM table_name;`

Here, column1, column2... are the fields (or columns) of a table (table_name).

However, if you want query all records, just use:

- `SELECT * FROM table_name;`

```
In [1]: %load_ext sql
```

1. Connet to the given database of demo.db3

```
In [3]: %sql sqlite:///data/demo.db3
```

If you do not remember the tables in the demo data, you can always use the follow command to query. Here we select the table of watershed_yearly as an example.

```
In [4]: %sql SELECT name FROM sqlite_master WHERE type='table'
```

* sqlite:///data/demo.db3
Done.

```
Out[4]:
```

name
rch
hru
sub
sed
watershed_yearly

ex04-Constrain the Number of Rows Returned by a SELECT Query

Often it is not convenient to show all rows on screen as you have to scroll. We can use SQLite **LIMIT** clause to constrain the number of rows returned by a query. This can be used for testing or to prevent consuming excessive resources if the query returns more information than expected.

The following illustrates the syntax of the LIMIT clause:

- SELECT column_list FROM table_name LIMIT row_count;

```
In [2]: %load_ext sql
```

1. Connet to the given database of demo.db3

```
In [3]: %sql sqlite:///data/demo.db3
```

2. Retrieving the first 10 rows of the table of watershed_yearly

Add the LIMIT clause to the SELECT statement to get exact 10 rows.

```
In [4]: %sql SELECT * From watershed_yearly LIMIT 10
```

```
* sqlite:///data/demo.db3
Done.
```

```
Out[4]:
```

YR	PREC_mm	SURQ_mm	LATQ_mm	GWQ_mm	PERCOLA_mm	TILEQ_mm	SW_mm	ET
1981	895.6051025390625	130.3574676513672	51.33439254760742	406.0739440917969	443.198486328125	0.0	18819.70703125	236.9830017089
1982	884.670654296875	92.21265411376953	50.34769058227539	367.41070556640625	424.6783752441406	0.0	19425.8359375	256.7908325195
1983	816.6605224609375	129.16310119628906	54.47308349609375	445.33306884765625	462.76324462890625	0.0	18476.1953125	237.7055511474
1984	867.5743408203125	95.57946014404297	46.948646545410156	347.2448425292969	408.5397033691406	0.0	17727.712890625	223.0897521972
1985	637.7255249023438	75.55760192871094	45.336883544921875	358.2953186035156	361.4852284921875	0.0	15840.5400390625	236.3213500976
1986	733.8412475585938	65.63016510009766	36.76259231567383	241.9388427734375	281.6683349609375	0.0	16059.4345703125	243.030563354
1987	1007.8944702148438	110.28632354736328	62.23741912841797	445.5394592285156	493.0043029785156	0.0	19541.869140625	256.957000732
1988	895.8466186523438	149.65235900878906	61.493255615234375	450.4956970214844	493.1164245605469	0.0	17205.849609375	220.839523315
1989	930.10546875	111.37128448486328	57.6666145324707	418.1727600097656	474.9247741699219	0.0	17915.3359375	208.5802612304
1990	751.4553833007812	102.00208282470703	54.310569763183594	409.75189208984375	435.1587829589844	0.0	16842.517578125	213.6323394775

Or we can simplify the above filter using the **IN** statement.

```
In [11]: %sql SELECT RCH, YR, MO, FLOW_INcms, FLOW_OUTcms From rch WHERE YR>2009 and RCH=10 and MO in (3, 6, 9, 12)
```

```
* sqlite:///data/demo.db3
Done.
```

```
Out[11]:
```

RCH	YR	MO	FLOW_INcms	FLOW_OUTcms
10	2010	3	3.4298956394195557	3.0846056938171387
10	2010	6	736.3016357421875	734.9920654296875
10	2010	9	218.40797424316406	218.24456787109375
10	2010	12	36.9558219909668	36.877464294433594

Or the months are **NOT** in 3, 6, 9, 12

```
In [12]: %sql SELECT RCH, YR, MO, FLOW_INcms, FLOW_OUTcms From rch WHERE YR>2009 and RCH=10 and MO NOT IN (3, 6, 9, 12)
```

```
* sqlite:///data/demo.db3
Done.
```

```
Out[12]:
```

RCH	YR	MO	FLOW_INcms	FLOW_OUTcms
10	2010	1	9.619468688964844	9.422823905944824
10	2010	2	6.129702091217041	5.864690780639648
10	2010	4	258.67010498046875	258.1203918457031
10	2010	5	1513.6510009765625	1512.5479736328125
10	2010	7	583.045654296875	582.3214721679688
10	2010	8	371.0841369628906	370.7648010253906
10	2010	10	123.48409271240234	123.36622619628906
10	2010	11	68.6158676147461	68.5250473022461

2.6 Filter with math operators

```
In [5]: %sql SELECT 12 * 4
* sqlite:///data/demo.db3
Done.

Out[5]: 12 * 4
48

In [6]: %sql SELECT 12 % 4
* sqlite:///data/demo.db3
Done.

Out[6]: 12 % 4
0

In [6]: %sql SELECT round(123.456, 2) as Rounded
* sqlite:///data/demo.db3
Done.

Out[6]: Rounded
123.46
```

3. Doing Math Across Table Columns

Take the table of watershed_monthly as an example

3.1 Check the table columns firstly.

```
In [8]: %sql SELECT * From watershed_monthly LIMIT 3
* sqlite:///data/demo.db3
Done.

Out[8]:
```

	YR	MO	PREC_mm	SURQ_mm	LATQ_mm	GWQ_mm	PERCOLA_mm	TILEQ_mm	SW_mm
1981	1	96.2901611328125	0.5159813761711121	0.4125469923019409	6.688113689422607	19.906705856323242	0.0	1854.2242431840625	6.8717
1981	2	160.22804260253906	3.346802234649658	0.6452788710594177	9.148776054382324	34.36812210083008	0.0	1766.2530517578125	9.1655
1981	3	136.6529083251953	3.8249943256378174	1.4813165664672852	18.518495559692383	34.367244720458984	0.0	1990.7535400390625	13.5204

功能描述:

1. 表结构与基础操作

代码通过 SQL 命令（如 SHOW TABLES 和 DESCRIBE）查询数据库中的表结构，获取字段、数据类型和约束条件。这些操作帮助用户快速了解数据库设计。同时，代码展示了创建表的过程，定义了字段的主键、外键及唯一性约束，为数据规范化存储奠定了基础。这部分内容适用于 MySQL 和 SQLite 等关系型数据库。

2. 数据检索与过滤

代码使用 SELECT 语句结合条件筛选（WHERE 子句）和行限制（LIMIT）实现精准的数据检索。例如，筛选教师表中出生日期早于某年的记录，或课程表中指定语言的课程。代码还展示了如何结合字段值动态运算生成新列。这些功能在 SQLite 和 MySQL 中同样适用，适合从大规模数据集中高效提取有用信息。

3. 数据运算与统计分析

代码通过聚合查询（GROUP BY 和 ORDER BY），对数据进行分组统计和结果排序。例如，按语言统计课程数量，或按开课时间排序课程列表。SQLite 同样支持这些操作，并在小型应用和轻量级分析场景中表现出色。这部分代码还展示了通过 SQL 函数处理表数据，为更复杂的数据分析提供了支持。

4. 联合查询与视图

代码利用 `JOIN` 操作实现了多表联合查询，例如通过课程表和教师表的连接获取每门课程的讲师信息。此外，代码还展示了如何创建和使用视图，将复杂查询逻辑封装为简洁的视图结构供后续调用。这些功能在 `MySQL` 和 `SQLite` 中均表现一致，但 `SQLite` 的轻量级特性更适合快速创建和测试这些关系。

5. 处理 `NULL` 值

代码专门演示了如何在数据库中处理 `NULL` 值，例如使用 `IS NULL` 或 `IS NOT NULL` 条件筛选记录，以及通过 `SQL` 函数将 `NULL` 替换为默认值。处理缺失值是数据库管理的重要环节，代码在 `MySQL` 和 `SQLite` 中都可以灵活适用，确保数据分析的完整性。

6. `SQLite` 的应用

代码特别展示了使用 `SQLite` 数据库进行数据存储和管理的能力。`SQLite` 是一种轻量级嵌入式数据库，适合小型项目和快速开发。代码中展示了如何创建时间序列数据库并存储数据，结合 `Pandas` 的功能，将时间序列数据从数据库中提取为 `DataFrame` 进行进一步处理。这种设计适合本地化时间序列分析任务，例如简单的日志管理或小型应用的数据存储。

7. 与 `Pandas` 的深度集成

代码展示了如何将查询结果直接加载为 `Pandas` 的 `DataFrame` 格式。这种结合不仅简化了数据操作，还方便用户进行进一步的分析和可视化。例如，将课程数据转为表格形式后，用户可以直接进行清洗、过滤或生成统计图表。这种 `MySQL` 和 `SQLite` 的结合方式，特别适合需要复杂分析的 `Python` 数据科学工作流。

总体而言，代码展示了从基础的数据库操作到高级分析的完整流程，涵盖了 `MySQL` 和 `SQLite` 的功能，尤其是 `SQLite` 在轻量级任务中的高效应用。通过 `Pandas` 的集成，代码进一步提升了数据分析能力，为数据科学、业务统计和应用开发提供了强有力的工具。`SQLite` 的灵活性和轻量化特性使其成为小型项目和快速开发的理想选择。

2. 实验二

部分截图：

```
In [5]: > def create_database(connection, query):
        cursor = connection.cursor()
        try:
            cursor.execute(query)
            print("Database created successfully")
        except Error as err:
            print(f"Error: '{err}'")

        create_database_query = "CREATE DATABASE school"
        create_database(connection, create_database_query)

        Database created successfully
```

2.3 - Modify Server Connection Function, Create Database Connection Function

Now that we've created a DB, let's modify our `create_server_connection` function to create a new function for connecting directly to that DB. This will prove more useful than just connecting to our server.

```
In [6]: > def create_db_connection(host_name, user_name, user_password, db_name):
        connection = None
        try:
            connection = mysql.connector.connect(
                host=host_name,
                user=user_name,
                passwd=user_password,
                database=db_name
            )
            print("MySQL Database connection successful")
        except Error as err:
            print(f"Error: '{err}'")

        return connection
```

2.4 - Define Query Execution Function

The final step of this stage is to create a function which will allow us to execute queries written in SQL. This is going to be extremely useful!

Again, we use `cursor.execute()` to execute our commands.

--- ~~FROM THE PREVIOUS CHAPTER~~ ---

The final step of this stage is to create a function which will allow us to execute queries written in SQL. This is going to be extremely useful!

Again, we use `cursor.execute()` to execute our commands.

```
In [7]: > def execute_query(connection, query):
        cursor = connection.cursor()
        try:
            cursor.execute(query)
            connection.commit()
            print("Query successful")
        except Error as err:
            print(f"Error: '{err}'")
```

3. Creating Tables

3.1 - Create Teacher Table

Now let's create our first table inside our DB, using our newly defined functions.

```
In [20]: > # Assign our SQL command to a python variable using triple quotes to create a multi-line string
        create_teacher_table = """
        CREATE TABLE teacher (
            teacher_id INT PRIMARY KEY,
            first_name VARCHAR(40) NOT NULL,
            last_name VARCHAR(40) NOT NULL,
            language_1 VARCHAR(3) NOT NULL,
            language_2 VARCHAR(3),
            dob DATE,
            tax_id INT UNIQUE,
            phone_no VARCHAR(20)
        );
        """

        connection = create_db_connection("localhost", "root", pw, db) # Connect to the Database
        execute_query(connection, create_teacher_table) # Execute our defined query

        MySQL Database connection successful
        Query successful
```

If we check in our MySQL terminal using `SHOW TABLES`; we see the following:

3.2 - Create Remaining Tables

Now let's create the rest of our tables.

```
In [21]: create_client_table = """
CREATE TABLE client (
    client_id INT PRIMARY KEY,
    client_name VARCHAR(40) NOT NULL,
    address VARCHAR(60) NOT NULL,
    industry VARCHAR(20)
);
"""

create_participant_table = """
CREATE TABLE participant (
    participant_id INT PRIMARY KEY,
    first_name VARCHAR(40) NOT NULL,
    last_name VARCHAR(40) NOT NULL,
    phone_no VARCHAR(20),
    client INT
);
"""

create_course_table = """
CREATE TABLE course (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(40) NOT NULL,
    language VARCHAR(3) NOT NULL,
    level VARCHAR(2),
    course_length_weeks INT,
    start_date DATE,
    in_school BOOLEAN,
    teacher INT,
    client INT
);
"""

connection = create_db_connection("localhost", "root", pw, db)
execute_query(connection, create_client_table)
execute_query(connection, create_participant_table)
execute_query(connection, create_course_table)

MySQL Database connection successful
```

3.3 - Define Foreign Key Relationships

Now altering the tables to create Foreign Key relationships (see the accompanying [SQL Tutorial Series](#) on [Towards Data Science](#) for the background on all of this), and creating our final table, takes_course

```
[22]: alter_participant = """
ALTER TABLE participant
ADD FOREIGN KEY(client)
REFERENCES client(client_id)
ON DELETE SET NULL;
"""

alter_course = """
ALTER TABLE course
ADD FOREIGN KEY(teacher)
REFERENCES teacher(teacher_id)
ON DELETE SET NULL;
"""

alter_course_again = """
ALTER TABLE course
ADD FOREIGN KEY(client)
REFERENCES client(client_id)
ON DELETE SET NULL;
"""

create_takescourse_table = """
CREATE TABLE takes_course (
    participant_id INT,
    course_id INT,
    PRIMARY KEY(participant_id, course_id),
    FOREIGN KEY(participant_id) REFERENCES participant(participant_id) ON DELETE CASCADE, -- it makes no sense to keep this relation when a
    FOREIGN KEY(course_id) REFERENCES course(course_id) ON DELETE CASCADE
);
"""

connection = create_db_connection("localhost", "root", pw, db)
execute_query(connection, alter_participant)
execute_query(connection, alter_course)
execute_query(connection, alter_course_again)
execute_query(connection, create_takescourse_table)
```

功能描述:

1. 数据库连接与创建

代码通过 `mysql.connector` 提供的 `connect()` 方法实现与 **MySQL** 服务器的连接。函数 `create_server_connection` 用于建立服务器连接，用户需要提供主机名、用户名和密码等信息。随后，函数 `create_database` 使用 `CREATE DATABASE SQL` 命令在服务器上创建名为 `school` 的数据库。在这之上，扩展的 `create_db_connection` 函数

允许用户直接连接到特定数据库，这为后续的表操作和数据管理提供了便利。

2. 表的创建与外键关系

代码通过 `CREATE TABLE` 命令定义了多个表结构，包括 `teacher`、`client`、`participant` 和 `course` 表。这些表中包含多种字段类型及约束条件，如主键、外键和唯一性约束。通过 `ALTER TABLE` 添加外键关系，确保了表之间的数据一致性。例如，`course` 表通过外键与 `teacher` 和 `client` 表关联，体现了清晰的数据层级和业务逻辑。这种设计为数据库的管理和查询提供了高效的组织形式。

3. 数据插入与批量操作

代码提供了多种数据插入方式，包括单条插入和批量插入。单条插入通过 `INSERT INTO` 实现，而批量插入使用了 `executemany` 方法。批量插入以参数化查询的形式接收多条记录，既提高了效率，也有效防止了 SQL 注入攻击。在实验中，数据涵盖教师、客户、参与者及课程等多种实体，展示了完整的数据填充流程。`execute_query` 函数在此处起到了统一执行 SQL 操作的作用，简化了插入逻辑。

4. 数据查询与格式化

函数 `read_query` 用于执行 SQL 查询，返回的结果可以直接打印或进行格式化处理。例如，查询教师的基本信息、课程的筛选结果，以及跨表数据的关联查询均使用该函数完成。代码展示了如何将查询结果转换为 Python 列表、列表的列表，甚至 Pandas DataFrame。尤其是与 Pandas 的结合，使得数据库数据可以被用于后续的数据分析和可视化操作。

5. 数据更新与删除

代码展示了如何通过 `UPDATE` 和 `DELETE` 语句更新或删除数据库中的记录。例如，更新客户地址时，使用 `WHERE` 子句确保只影响目标记录，避免误操作。删除操作同样采用了安全的方式，通过条件筛选特定记录进行删除。代码强调了操作的谨慎性，并支持用户对已删除数据进行恢复操作，以满足更复杂的应用需求。

6. 高级功能与安全性

代码通过 `executemany` 支持批量操作，并使用参数化查询以确保安全性，防止 SQL 注入。这种方法特别适合需要从外部数据源批量更新或插入数据的场景。同时，代码还展示了如何定义并操作外键关系，使得数据表的设计更加严谨，便于维护。

总体来看，从数据库创建到管理数据的完整流程，结合 Python 和 MySQL 的功能，适合实现数据分析、系统开发等多种应用场景。

3. 实验三

部分截图：

jupyter Exercises 最新检查点: 2024/10/14 (已自动保存)

File Edit View Insert Cell Kernel Widgets Help

Python 3 (ipykernel)

Out[6]:

	month	unique_species
0	1	30
1	2	32
2	3	28
3	4	33
4	5	29
5	6	27
6	7	32
7	8	30
8	9	28
9	10	33
10	11	28
11	12	28

Q2

print the increase or decrease in the number of unique species observed per month

```
In [7]: # method 1: list operation

In [8]: species_counts = list(species_month_count['unique_species'])

In [9]: for i in range(len(species_counts)-1):
        print(i+1, species_counts[i+1] - species_counts[i])

1 2
2 -4
3 5
4 -4
5 -2
6 5
7 -2
8 -2
9 5
10 -5
11 0
```

jupyter Python-Pandas 最新检查点: 2024/10/14 (已自动保存)

File Edit View Insert Cell Kernel Widgets Help

Python 3 (ipykernel)

```
In [2]: # Merging, sorting, and querying using Pandas
#
# Objectives:
# Use pandas operations to perform common SQL operations

In [6]: import pandas as pd

In [21]: surveys_df = pd.read_csv("data/surveys.csv")
species_df = pd.read_csv("data/species.csv")

In [8]: # you can get the head and tail, similar to the LIMIT clause in SQL

In [37]: surveys_df.head()

Out[37]:
```

	record_id	month	day	year	plot_id	species_id	sex	hindfoot_length	weight
0	1	7	16	1977	2	NL	M	32.0	NaN
1	2	7	16	1977	3	NL	M	33.0	NaN
2	3	7	16	1977	2	DM	F	37.0	NaN
3	4	7	16	1977	7	DM	M	36.0	NaN
4	5	7	16	1977	3	DM	M	35.0	NaN

```
In [38]: surveys_df.tail()

Out[38]:
```

	record_id	month	day	year	plot_id	species_id	sex	hindfoot_length	weight
35544	35545	12	31	2002	15	AH	NaN	NaN	NaN
35545	35546	12	31	2002	15	AH	NaN	NaN	NaN
35546	35547	12	31	2002	10	RM	F	15.0	14.0
35547	35548	12	31	2002	7	DO	M	36.0	51.0
35548	35549	12	31	2002	5	NaN	NaN	NaN	NaN

```
In [9]: # to select only certain columns
# similar to SELECT species_id, weight FROM..

In [10]: surveys_df[['species_id', 'weight']].head()
```

```
jupyter Python-SQLite 最新检查点: 2024/09/27 (已自动保存) Logout

File Edit View Insert Cell Kernel Widgets Help 不可信 Python 3 (ipykernel)

In [6]: # now that you can retrieve each row, you are able to access data within each row
# as a tuple.
# here, we retrieve the first and third element for each row

In [7]: for row in c.execute('SELECT * FROM species LIMIT 5'):
        print(row[0], row[2])

AB bilineata
AH harrisi
AS savannarum
BA taylori
CB brunneicapillus

In [8]: # getting information about the sqlite database
# without a browser (such as db browser for sqlite), it's harder to get info about the
# tables and column names.
# we can use query the sqlite schema to all tables and data elements

In [9]: for row in c.execute("SELECT * FROM sqlite_schema"):
        print(row)

('table', 'surveys', 'surveys', 2, 'CREATE TABLE surveys (\n\trecord_id BIGINT, \n\tmonth BIGINT, \n\tday BIGINT, \n\tyear BIGINT, \n\tplo
ot_id BIGINT, \n\tspecies_id TEXT, \n\tsex TEXT, \n\thindfoot_length FLOAT, \n\tweight FLOAT\n)')
('table', 'species', 'species', 241, 'CREATE TABLE species (\n\tspecies_id TEXT, \n\tgenus TEXT, \n\tspecies TEXT, \n\ttaxa TEXT\n)')
('table', 'plots', 'plots', 242, 'CREATE TABLE plots (\n\tplot_id BIGINT, \n\tplot_type TEXT\n)')

In [10]: # to get detailed information about just one table, we can use pragma

In [11]: for row in c.execute('pragma table_info("surveys")'):
        print(row)

(0, 'record_id', 'BIGINT', 0, None, 0)
(1, 'month', 'BIGINT', 0, None, 0)
(2, 'day', 'BIGINT', 0, None, 0)
(3, 'year', 'BIGINT', 0, None, 0)
(4, 'plot_id', 'BIGINT', 0, None, 0)
(5, 'species_id', 'TEXT', 0, None, 0)
(6, 'sex', 'TEXT', 0, None, 0)
(7, 'hindfoot_length', 'FLOAT', 0, None, 0)
(8, 'weight', 'FLOAT', 0, None, 0)
```

功能描述:

1. SQLite 数据库的连接与查询

首先使用 Python 的 `sqlite3` 模块连接 SQLite 数据库。在此过程中,我创建了一个名为 `portal_mammals.sqlite` 的数据库,并尝试执行以下操作:读取表结构:通过 `SELECT name FROM sqlite_master` 查询数据库中的所有表。执行简单查询:如查询 `surveys` 表中的所有数据。处理结果:将查询结果转为 Python 的元组列表格式,并使用 `Pandas` 转为 `DataFrame` 以便后续处理。在这一部分中,学会了如何通过 Python 灵活管理 SQLite 数据库,并体验了 SQL 查询的高效性。

2. 数据清洗与处理

加载了实验提供的 CSV 文件 (`species.csv`、`surveys.csv` 和 `plots.csv`),并利用 `Pandas` 对数据进行了清洗和合并操作。数据清洗:删除数据中的重复行和缺失值 (`drop_duplicates` 和 `fillna`)。将时间字段标准化为一致格式。数据合并:使用 `Pandas` 的 `merge` 方法,将 `species` 表和 `surveys` 表通过公共字段 `species_id` 合并。将合并后的数据进一步与 `plots` 表关联,生成一张包含全面信息的整合表。这一过程中,切实感受到了 `Pandas` 在数据处理方面的强大功能,特别是在清洗和合并多源数据时的简便性。

3. SQL 与 Pandas 的结合

在实验的核心部分，结合了 SQL 和 Pandas 的优点进行分析操作：**Pandasql 的使用**：利用 `pandasql` 模块直接在 `Pandas DataFrame` 上运行 SQL 查询，而无需额外建立数据库。这种方式让我能够快速执行复杂查询，比如筛选特定物种的栖息地信息，统计不同年份的调查记录数等。**高级 SQL 查询**：使用子查询和聚合函数（如 `GROUP BY` 和 `HAVING`）分析数据分布。**创建视图**以保存常用查询逻辑，便于后续复用。**数据导出**：将分析结果通过 `Pandas` 保存为新的 `CSV` 文件或写入 `SQLite` 数据库，方便数据的后续使用。通过这些操作，体会到 SQL 的强大查询能力和 `Pandas` 的灵活数据操作在解决实际问题时是如何高效结合的。

4. 数据分析与可视化

在数据分析部分，对合并后的数据进行了多角度的探索。**分组与排序**：统计每种动物的出现频率，并按降序排列。**时间序列分析**：提取出不同年份的调查数据，并绘制折线图观察变化趋势。**可视化结果**：结合 `Matplotlib` 和 `Seaborn`，将分析结果生成直观的可视化图表，例如柱状图、散点图和时间序列图。

通过第三个实验，从数据管理、清洗、到分析的完整流程中，不仅强化了对 SQL 和 `Pandas` 的理解，还实践了如何高效结合两者处理实际数据。具体收获如下：**工具结合的优势**：SQL 在查询效率上占优，而 `Pandas` 在灵活性和扩展性上更强，将二者结合可以取长补短。**解决实际问题的能力**：实验数据中包含缺失值和不一致格式，通过清洗和规范化操作提升了数据质量，为后续分析打下了基础。**掌握新的技术**：通过实践 `pandasql` 模块和 `SQLite`，拓展了对数据分析的工具链，能更从容应对多源数据的整合任务。

四、实验总结

通过本次实验，我全面掌握了 `Python`、`Pandas` 和 SQL 在数据处理与分析中的应用。这次实践让我切实感受到 SQL 在数据查询中的高效性、`Pandas` 在数据清洗与处理中的灵活性，以及 `Python` 在整合多种工具中的强大能力。从 `SQLite` 的数据库操作到结合 `pandasql` 直接在 `Pandas DataFrame` 上运行 SQL 查询，再到通过 `Matplotlib` 和 `Seaborn` 生成可视化分析结果，每一步都展现了多工具协作的优势。实验中解决了数据清洗、表合并、复杂查询以及数据可视化等多维度问题，不仅提升了数据处理效率，也强化了对数据质量管理的理解。此次实验让我深刻认识到，面对实际数据任务时，单一工具可能无法满足需求，而灵活结合多种工具才能实现更高效、更全面的解决方案。未来，我将继续探索这些工具在实际项目中的应用，进一步提升自己的数据分析能力。